

Real-Time Webhook Event Processing Workflow

Lesson 5 Assessment — Documentation Package

Platform: n8n

Workflow: New User Webhook Event Processor

This document describes the webhook architecture, request inspection, X-Webhook-Token validation, authorization routing, payload normalization, HTTP response design, deployment behavior, security considerations, troubleshooting notes, and test plan.

Assessment focus: reliable event-driven webhook processing with authorized and unauthorized request handling.

1. Assessment Overview

The project implements a POST webhook endpoint that receives a new-user event, inspects the incoming request, validates a custom X-Webhook-Token header, routes authorized and unauthorized requests, normalizes user data, and returns a meaningful HTTP response to the calling client.

Assessment objectives covered

Area	Implementation
Webhook intake	POST webhook with path new-user and responseNode mode.
Request inspection	Incoming headers, query parameters, params, and body are available to downstream nodes.
Authentication	X-Webhook-Token is checked before processing the user profile.
Authorization routing	IF node sends valid requests to processing and invalid/missing tokens to 403 response.
Normalization	Email is trimmed and lowercased; name whitespace is normalized; receivedAt is generated.
Responses	Authorized requests return HTTP 200 JSON; unauthorized requests return HTTP 403 JSON.
Documentation	Workflow uses descriptive node names and is suitable for repository/LMS documentation.

The exported workflow JSON confirms the webhook path is **new-user**, uses POST, and is configured with **responseMode: responseNode**. The workflow is currently marked inactive in the exported JSON, so production behavior should be verified after activation/publishing in n8n.

2. Workflow Architecture

The workflow follows a simple event-driven sequence. The webhook receives the request and passes it to an authorization gate. Only the authorized branch reaches normalization and the 200 response; the rejection branch returns 403 immediately.

NEW USER CLIENT		
POST /new-user	→	New User Webhook (POST)
	↓	
Validate X-Webhook-Token		
INVALID / MISSING	← IF →	VALID TOKEN
Respond 403 Forbidden		Normalize User Profile
		↓
		Respond 200 Success

Node	Purpose
New User Webhook (POST)	Receives POST requests at the new-user webhook path and exposes request data.
Validate X-Webhook-Token	Compares the incoming x-webhook-token header against the configured expected value.
Normalize User Profile	Normalizes name/email fields and creates receivedAt timestamp.
Respond 200 Success	Returns a JSON success response with normalized user data and HTTP 200.
Respond 403 Forbidden	Returns a JSON error response for invalid or missing tokens with HTTP 403.

3. Node Configuration Documentation

3.1 Webhook Trigger

The exported workflow defines a POST webhook with the path **new-user**. The response mode is **responseNode**, meaning a Respond to Webhook node controls the HTTP response.

Configuration summary

Property	Configured value
HTTP Method	POST
Path	new-user
Response Mode	responseNode
Purpose	Receive real-time new-user events and return custom HTTP responses.

3.2 Token Validation

The IF node reads the request header using the expression `$json.headers["x-webhook-token"]` and compares it with the expected token value. The exported JSON contains a placeholder for the expected value; it must be replaced/configured in the n8n workflow before production use.

3.3 Payload Normalization

The Set/Edit Fields node creates three normalized fields: name, email, and receivedAt. The email expression trims whitespace and converts the value to lowercase. The timestamp uses the n8n current-time expression and is converted to ISO format.

Implementation note: the exported name expression uses `.toLowerCase()`. This is not a standard JavaScript String method, so the name-normalization expression should be tested in n8n and replaced with a supported expression if it errors. This note is based on the exported workflow itself.

3.4 HTTP Responses

Condition	Status	Response
Valid X-Webhook-Token	200	JSON success object containing status, message, normalized name, email, and receivedAt
Invalid or missing token	403	JSON error object stating that the X-Webhook-Token is invalid or missing.

4. Security and Deployment Review

The workflow uses a custom request header as a lightweight authorization mechanism. The design separates authorization from business processing so rejected requests do not reach the normalization/success branch.

Risk / concern	Recommended practice
Hardcoded production token	Do not expose a real secret in documentation, screenshots, GitHub, or workflow expressions. Use secrets.
Token leakage in logs	Avoid logging the raw X-Webhook-Token value. Inspect only whether validation succeeded.
Test endpoint exposure	Use the n8n test URL only while listening/testing; use the production URL after activation/publishing.
Public repository	Never commit real webhook secrets, API keys, passwords, or personal access tokens.
Replay/abuse	For production systems, consider stronger authentication, rate limiting, request signing, timestamp/nonce.

Troubleshooting notes

Problem	Check / resolution
Webhook does not receive request	Confirm HTTP method, path, workflow activation, and whether the correct test or production URL is being used.
Always returns 403	Check the exact header name/value, whitespace, casing behavior, and the expected-token configuration.
Name normalization fails	Test the name expression; replace unsupported .toLowerCase() logic with a valid n8n expression.
200 response not returned	Confirm the authorized branch reaches Respond 200 Success and that responseMode is responseNodeSuccess.
Production endpoint unavailable	Activate/publish the workflow and use the production webhook URL rather than the test URL.

5. Authentication / Webhook Test Plan

The following tests are designed to provide LMS evidence for authorized routing, unauthorized routing, normalization, and deployment behavior.

ID	Precondition	Test	Expected result
T01	Workflow available in test mode	POST valid JSON with correct X-Webhook-Token.	Request reaches normalization and returns HTTP 200.
T02	Workflow available in test mode	POST valid JSON with wrong token.	Request is routed to rejection branch and returns HTTP 403.
T03	Workflow available in test mode	POST without X-Webhook-Token.	Request is rejected with HTTP 403.
T04	Authorized request	Use mixed-case/whitespace email.	Email is trimmed and lowercased in the normalized output.
T05	Authorized request	Use a name with repeated whitespace.	Whitespace is normalized; name formatting should be verified at production.
T06	Workflow activated	Call production webhook URL.	Production endpoint processes the request and returns the configuration.
T07	Workflow inactive	Attempt production endpoint before activation.	Endpoint should not be treated as a production-ready endpoint until activated.

Sample cURL test commands

Replace the URL and token placeholder with your n8n test/production endpoint and configured value. Do not publish a real token in GitHub.

```
curl -X POST "/new-user" ^ -H "Content-Type: application/json" ^ -H "X-Webhook-Token: " ^ -d
{"name":"Jane Doe","email":"" JANE.DOE@EXAMPLE.COM ""}

curl -X POST "/new-user" ^ -H "Content-Type: application/json" ^ -H "X-Webhook-Token: WRONG_TOKEN" ^ -d
{"name":"Jane Doe","email":"","jane@example.com"}"
```

6. Submission Package

Deliverable	Status / action
Loom/YouTube demonstration	Record authorized + unauthorized tests, normalization, 200/403 responses, and test vs production endpoint.
Public GitHub repository	Include workflow JSON, this PDF, screenshots, test evidence, setup instructions, and sample cURL commands.
Workflow JSON	Use the exported New User Webhook Event Processor JSON.
Screenshots	Include the n8n canvas and execution outputs for both success and rejection paths.
LMS Description	Mention challenges, resolutions, assumptions, limitations, security considerations, and improvements.

7. Implementation Notes for Assessor

The exported n8n workflow contains five core nodes: New User Webhook (POST), Validate X-Webhook-Token, Normalize User Profile, Respond 200 Success, and Respond 403 Forbidden. The connections implement a clear authorized/unauthorized split. The workflow JSON also records response headers as application/json for the custom responses.

The expected X-Webhook-Token value is represented by a placeholder in the exported JSON. This is intentional for safe source sharing and must be configured with a real test value inside the n8n environment before execution.

Known limitation: the current name-normalization expression calls `.toTitleCase()`. This should be validated or replaced with a supported expression before claiming the name-formatting requirement is fully production-ready.

8. Final Checklist

- POST webhook configured with path new-user
- Request body, headers, and query parameters inspected
- X-Webhook-Token validation configured
- Authorized path reaches normalization and HTTP 200
- Unauthorized path returns HTTP 403
- Email normalization tested
- Name normalization expression tested/fixed if necessary
- Test webhook URL verified
- Production webhook URL verified after activation
- Workflow JSON exported
- Screenshots captured
- GitHub repository updated
- Loom/YouTube video recorded
- PDF documentation uploaded to LMS